PICTUREHOUSE JAIPUR

Comprehensive Project Report

Architecture, technology choices, object-oriented design, database model, constraints, normalization, and feature-by-feature operation

Prepared from the current project source code and configuration

Project snapshot: 22 October 2026 | Application: Jaipur-first cinema booking and shared-ride prototype

This report describes current implementation, labels prototype assumptions, and distinguishes database-enforced rules from application-level checks. Third-party technical references are listed in Appendix B.

Scope note: application.yml defaults to a file-backed H2 database. MySQL Connector/J is present as a runtime option, but Workbench connecting successfully does not prove this backend process is using MySQL. The running backend datasource URL determines the active database.

Contents

- 1. Executive overview
- 2. Product goals and users
- 3. Architecture and request flow
- 4. Technology stack and rationale
- 5. Object-oriented design in the implementation
- 6. Feature operation, end to end
- 7. Database model and entity inventory
- 8. Keys, constraints, validation, and transactions
- 9. Normalization review and expansion toward BCNF
- 10. MySQL adoption and operations
- 11. Security, reliability, and known limitations
- 12. Recommended next steps
- Appendix A. Logical ER diagram
- Appendix B. Source and research references

Evidence standard: project-specific claims were checked against repository files. Recommendations are labeled as future work rather than claims that a feature exists.

1. Executive overview

Picturehouse Jaipur is a single-page cinema discovery and booking app backed by a Java REST API. A customer browses Jaipur movie listings, chooses a date and show, selects seats, pays through Razorpay Test Mode, and views a QR ticket. Customers can also post or request shared rides to a cinema. A separate theatre-owner portal lets approved representatives view show-level sales and manage movies and showtimes. Clerk provides sign-in and issues tokens validated by the backend.

The architecture uses React for presentation and interaction; Spring Boot for HTTP APIs, security, validation, payment verification, business workflows, and persistence; and JPA/Hibernate to map Java entities to relational tables. The HTTP API is stateless. External credentials are configured through backend environment variables.

The repository has 15 JPA entities/table mappings. The data model uses generated numeric primary keys, foreign-key relationships for core cinema and ride records, selected unique constraints, and selected indexes. Some payment/ticket rows store immutable purchase snapshots intentionally. The project has no checked-in SQL migration tool; Hibernate currently uses ddl-auto=update.

Current database state: application.yml defaults to H2. MySQL Connector/J and PostgreSQL JDBC drivers are available. MySQL is only selected when backend datasource environment variables target MySQL. A Workbench connection test validates MySQL access, not the datasource selected by the running Java API.

At a glance

Area	Current implementation
Frontend	React/JSX, JavaScript, Vite, CSS, Clerk React SDK.
Backend	Java 21, Spring Boot 3.5.6, Spring MVC, Spring Security JWT resource server, Bean Validation, Spring Data JPA/Hibernate.
Database	Relational JPA model; file H2 default; MySQL Connector/J and PostgreSQL driver included.
Integrations	Clerk, Razorpay Orders/Payments API in test mode, SMTP email, ZXing QR, WebSocket/STOMP configuration.
Build	npm with package-lock; Maven backend.

2. Product goals and users

2.1 Customer

The customer journey reduces the steps between discovering a film and arriving at the cinema. Customers search movies, browse venues and showtimes, select seats, pay, retrieve a QR ticket, and resend the ticket email. Ride together helps moviegoers find or post shared transport for a selected date. Clerk protects ticket ownership and user-specific actions.

2.2 Theatre representative

An authenticated user submits a request to represent a theatre. The backend stores a membership keyed by Clerk subject and theatre, with an approval flag. Only approved memberships authorize management. Owners see bookings and gross ticket sales by show, and can add/update movies and showtimes. Current approval is controlled by administrator Clerk IDs. A theatre-branded Gmail address is a possible naming convention but is not implemented and does not by itself prove venue ownership.

2.3 Administrator

Admin authorization is currently an environment list ADMIN_CLERK_USER_IDS, rather than an independent identity provider role. That allowlist protects pending-request and approval endpoints. A submitted membership is not access until approved.

2.4 Prototype boundary

Development catalogue entries are fictional and seeded only when the database has no movies or theatres. README warns that real theatre information must replace sample listings before accepting real bookings. Razorpay test credentials are explicitly required. Production still needs real catalogue operations, database migrations, payment reconciliation, privacy controls, monitoring, and gate-side QR redemption.

3. Architecture and request flow

The browser renders the UI and makes HTTP requests. Public catalogue routes are readable without sign-in. Routes under /api require a valid bearer token except GET /api/rides; health and WebSocket handshake are public. The frontend obtains a Clerk token and adds Authorization: Bearer. Spring Security validates JWT signature and claims using Clerk JWKS, checks configured issuer and an allowed azp origin, and exposes the principal to controllers. Repositories use Hibernate/JPA to query the configured relational database.

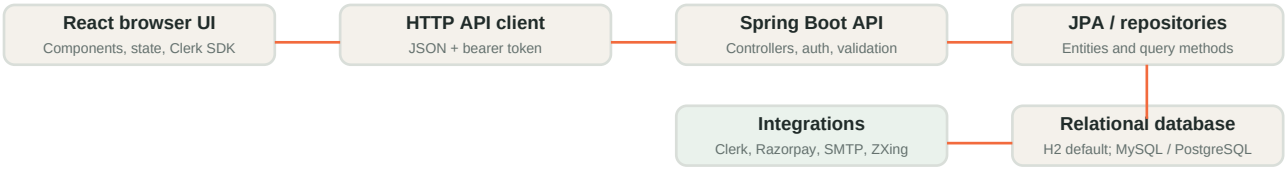

Figure 1. Runtime layers. H2 is configured as default; environment datasource values select MySQL or PostgreSQL.

3.1 Layer responsibilities

- **UI:** display and interaction state. It is not trusted to decide whether a payment is real or a person owns a theatre.
- **API/controller:** HTTP mapping, DTO validation, authorization checks, response shaping, and workflow orchestration.
- **Domain/persistence:** concepts such as Showtime, TicketBooking, RidePost and TheatreOwnerMembership, mapped through repositories.
- **Integration:** Razorpay calls, SMTP delivery, JWT validation and QR creation.
- **Transaction:** seat booking and ticket creation are persisted as a transaction, reducing inconsistent partial database updates.

3.2 Architectural tradeoffs

This is a modular monolith: one backend runs API, persistence, integrations and WebSocket configuration. That simplifies local development and keeps ticket/seat writes inside one database transaction. A message queue, email worker/outbox, shared cache, observability and distributed services may help at larger scale, but are not required to understand today's design.

4. Technology stack and rationale

4.1 React and JavaScript/JSX

React function components such as App, RideBoard and OwnerPortal make UI from state. Hooks keep selected film/date/seats, form data, modal visibility and API loading status. This is useful for UI states that change as users search, select showtimes, sign in and receive API results. React describes components as units for building UI and state as component memory for interaction [1]. The code uses JavaScript/JSX rather than TypeScript, which lowers initial setup but sacrifices compile-time DTO type checking.

4.2 Vite

Vite provides the local development server, React plugin integration, hot module replacement and production build. Its development server serves source modules on demand and updates changed modules; its production build emits optimized static assets [2]. The npm scripts are dev, build, preview and check (vite build). Current package.json uses “latest” for React/Vite-related dependencies; package-lock improves reproducibility, but floating manifest versions should be replaced with deliberate version ranges.

4.3 Java 21 and Spring Boot/Spring MVC

The backend uses Java 21 and Spring Boot 3.5.6. Spring Boot brings embedded server startup, dependency injection, externalized configuration and starter integrations. Spring MVC exposes annotated REST controllers. This fits a backend needing authentication, relational persistence, validation, email, web sockets and outbound payment HTTP calls.

4.4 Spring Data JPA, Hibernate and JDBC

JPA annotations map entities and relations to database tables; Hibernate is the persistence provider. Spring Data repository interfaces generate routine CRUD and derived queries, reducing hand-written data-access boilerplate [3]. Runtime drivers for H2, MySQL and PostgreSQL are included. The selected datasource URL, driver and credentials determine the live database. Schema details can still vary across SQL engines and should be validated against the target MySQL DDL.

4.5 Clerk and Spring Security

Clerk handles sign-in and issues JWT session tokens. Spring Security is configured as a JWT resource server. NimbusJwtDecoder fetches public verification keys from configured JWKS; validators check issuer/time and allowed azp origin. Spring Security resource-server support verifies JWT signatures and configured claims [4]. Authorization is also object-specific: a signed-in user only manages a theatre with approved membership, and the admin allowlist gates approval.

4.6 MySQL

MySQL is a practical target for this relational booking domain: it supports transactions and foreign keys, is widely deployed, and Workbench makes schema inspection accessible. InnoDB is the intended transactional engine; MySQL enforces primary/unique keys and foreign-key relationships [5][6]. The project ships Connector/J, but its current YAML default is H2 unless environment overrides point to MySQL.

4.7 Razorpay, SMTP, QR and WebSocket

RazorpayClient calls the Orders and Payments APIs server-side using Basic authentication. The browser uses Razorpay Checkout after an order is created. The backend verifies HMAC-SHA256 checkout signature, fetches payment from provider, and checks order, amount, INR currency and captured status before reserving seats. ZXing encodes ticket code as QR. Spring Mail sends confirmation email with QR image attachment. SMTP and Razorpay secrets are backend environment values. STOMP/simple-broker setup exists for /ws; current ride-chat feature stores/messages through REST operations, so the broker config alone does not mean all conversations are real-time.

4.8 Build tools

npm and package-lock manage JavaScript packages; Maven and the Spring Boot parent manage Java dependencies and run the API. Separate tools align with the language ecosystems.

5. Object-oriented design in the implementation

OOP groups related state and behavior into collaborating objects. Java concepts include classes, objects, encapsulation, inheritance, interfaces and packages [7]. OOP is primarily in the Java backend. The frontend uses function components, so it should not be called class-based OOP.

5.1 Classes and objects

Movie, Theatre, Screen, Showtime, ShowtimeSeat, TicketBooking, RidePost, RideJoinRequest, RideChatMessage and TheatreOwnerMembership are entity classes; records represent API request/response shapes. Entity instances model domain facts and relationships. Enums such as BookingStatus, Role and SeatCategory limit values to named choices.

5.2 Encapsulation

Entity fields are private; many classes expose getters but avoid public setters for state transitions. Methods such as Booking.confirm, Booking.cancel, RidePost.reserveSeat and TheatreOwnerMembership.approve express behavior and guard some invariants. This reduces arbitrary state changes. Some policy remains in controllers (for example seat price and owner authorization); an application service/domain layer could consolidate rules as the system grows.

5.3 Abstraction and interfaces

PaymentGateway and TicketDeliveryGateway define operations without requiring callers to know provider details. This is an abstraction seam: the interface expresses capability, and a concrete adapter handles integration specifics. RazorpayClient currently implements Razorpay operations; TicketEmailService handles current email delivery. WhatsApp appears as an interface operation but no live implementation exists in the ticket flow.

5.4 Polymorphism

Interface-based implementations make it possible to inject a different payment or delivery provider behind the same contract. That is the intended polymorphic extension point. There is currently no multi-provider runtime switching, so this is extensibility rather than a feature users can choose today.

5.5 Inheritance is intentionally limited

There is no custom inheritance tree for customer/owner/admin. These roles are represented through enums, authenticated Clerk subjects and membership/allowlist authorization. Domain objects are connected by associations/composition rather than subclassing.

5.6 Composition, association and layering

A Showtime refers to Movie and Screen; Screen refers to Theatre; ShowtimeSeat refers to Showtime; TheatreMovie links venue and movie; RideJoinRequest belongs to RidePost; RideChatMessage belongs to a request. These associations reflect real ownership and lifecycle. Controllers receive collaborators by constructor injection. A good next step is an explicit service layer for booking, ride coordination, ownership and ticket-email workflows, keeping controllers thin and external side effects out of entity objects.

5.7 Concurrency control

Showtime and RidePost use @Version optimistic locking. The database also uniquely constrains (showtime_id, seat_code). Payment verification requests seat locks, rechecks rows, then marks them booked and writes a ticket in a transaction. These mechanisms help avoid conflicting updates and duplicate inventory, but concurrency behavior must be validated on the actual production database.

6. Feature operation, end to end

6.1 Catalogue, date selection and discovery

Outside the prod profile, DevelopmentCatalogSeeder runs at startup. If movies or theatres already exist, it does nothing. On a blank schema it creates four fictional movies, three Jaipur theatres, a screen per venue, and shows for today and the next seven calendar dates. It creates 48 seat rows per show, some prebooked for demonstration. CatalogController uses Asia/Kolkata current time and returns on-sale shows until the start of day +8. The frontend date picker exposes eight dates from today. The API is authoritative, so a date with no actual showtimes correctly shows an empty state.

App fetches /catalog, holds film/date/search in state, filters titles locally and builds venue/showtime cards. Catalog responses include a notice that the listings are development data.

6.2 Seat selection and pricing

On choosing a show, the client opens a seat dialog and fetches /catalog/showtimes/{id}/seats. It refreshes every 2.5 seconds while open and disables booked seats. UI price is shown by row: front Rs 250, middle Rs 300, recliner Rs 500. The server does not trust browser totals: it validates selected seat pattern, distinctness and maximum eight, confirms the seats belong to the show, checks availability and recomputes amount in paise.

6.3 Identity and access

Clerk provides sign-in and token acquisition. clerkApiFetch attaches a bearer token. SecurityConfig exposes catalog and ride search publicly and requires authentication on other /api routes. It validates issuer, token time and the configured allowed origin claim. Ticket lookups are scoped to JWT subject. Email resend additionally needs the primary email claim in the Clerk session token.

6.4 Payment and confirmed booking

- Customer chooses show and seats; sign-in is required before pay.
- Frontend submits showtime ID and seats to POST /api/payments/razorpay/orders. Backend verifies unique valid seats, computes price, calls Razorpay Orders API, and stores provider order plus customer and purchase snapshot.
- Razorpay checkout returns order ID, payment ID and signature. Frontend submits them to /verify.
- Backend calculates expected HMAC-SHA256 using secret and constant-time comparison, then fetches provider payment and checks exact order, amount, INR currency and captured status.
- Backend locks selected seat rows and rechecks availability. If seats are free, it marks them booked, marks the order captured, and saves TicketBooking in one transaction.
- If payment has been captured but seats cannot be reserved, code attempts a refund instead of issuing a duplicate ticket.
- Response includes booking reference and ticket code. The user can view My Tickets and resend ticket email.

Payment is not one atomic transaction across Razorpay and MySQL. The current code validates signature and fetches provider status, but README lists webhooks as future work. Before live usage, add signed webhooks, idempotent reconciliation, refund lifecycle handling and durable email outbox processing.

6.5 QR ticket and ticket ownership

TicketBooking records booking reference, unique ticket code, order/payment identifiers, Clerk user, showtime ID, show/movie/theatre/seat/amount snapshot, creation time and email-sent timestamp. Ticket responses contain a QR data URL generated by ZXing. Email attaches a QR PNG. Current README says actual theatre-gate QR redemption remains future scope. A production verifier must look up ticket, validate show/state and enforce single-use redemption; a static QR alone is not a gate-control workflow.

6.6 Ride together

RideBoard defaults to today and allows dates up to 90 days. It loads active rides for that date and the public movie/theatre catalogue for the posting form. Posting requires sign-in. The submitted ride records pickup area, theatre/movie/show text, departure, seat count, contribution, date, Clerk subject and display name. The backend validates date range and departure consistency before saving RidePost. Public GET /api/rides filters by date and active state. A passenger can create a RideJoinRequest; the post author can accept/decline; persisted chat messages are tied to the join request. A unique ride/requester constraint prevents a duplicate join request. There is also an older CarpoolRide entity, but the current board uses the RidePost model.

6.7 Owner dashboard and management

A signed-in user asks for access to a theatre. The backend creates or returns one membership per Clerk user/theatre. Dashboard requires approved membership, queries on-sale showtimes for assigned theatres, and totals TicketBooking count and amount by show and across theatres. This is gross ticket receipts, not net revenue, payout, fees or accounting. Owners can add a movie-to-theatre assignment, create future showtimes within the 90-day limit, generate a 48-seat inventory, update showtime, and edit movie details when the movie is not shared with other theatres. Each endpoint checks approved membership for the target theatre. An admin list configured by Clerk subject IDs can approve pending requests.

A branded Gmail address alone is not ownership proof. To remove manual approval safely, a future onboarding process could verify control of an official theatre domain, require theatre documents/contract, send a one-time challenge to a pre-verified business contact, record approval evidence, and retain an audited recovery/transfer process.

6.8 Ticket email

TicketController finds the booking only if it belongs to the current Clerk user. It requires an email claim, calls TicketEmailService, sends an escaped HTML email and attached entry QR, then records email_sent_at. Synchronous SMTP means an SMTP delay/failure is felt in the API request; a queue/outbox would improve retry handling.

6.9 Search, theme, feedback

The App component holds search, current view, theme, date, selected film/show/seat, and modal state. Movie search filters titles client-side. Theme toggling changes a CSS class. Inline notices and toasts report API outcomes. Theme preference is not currently persisted.

7. Database model and entity inventory

The 15 listed JPA classes map to the tables visible in the user's Workbench screenshot. Every entity declares an IDENTITY-generated numeric primary key. Exact SQL column types, generated constraint names, delete rules and indexes depend on the selected Hibernate dialect and database. Current source annotations express intended mappings; no versioned SQL migration is checked in.

Table / entity	Purpose	Keys and notable integrity
accounts / Account	Profile-like email, mobile, display name, role and verification flags.	PK id; unique email/mobile. Clerk is identity for current flows; not every flow uses this table.
movies / Movie	Film title, language, genre, certification, duration, synopsis, image and active flag.	PK id; @NotBlank title/language; links to showtimes and venue assignment.
theatres / Theatre	Venue name, area/address/city, type, parking and active.	PK id; @NotBlank name, area, address.
screens / Screen	Named screen and seat count within a venue.	PK id; FK theatre_id; unique (theatre_id,name).
showtimes / Showtime	Movie/screen schedule, start, format, on-sale and version.	PK id; FKs movie and screen; starts_at index; @Version.
showtime_seats / ShowtimeSeat	Per-show seat code and booked flag.	PK id; FK showtime_id; unique (showtime_id,seat_code).
theatre_movies / TheatreMovie	Movie-to-theatre many-to-many link.	PK id; FKs theatre_id and movie_id; unique pair.
theatre_owner_memberships / TheatreOwnerMembership	Clerk user's access request/membership, approval and requestedAt.	PK id; FK theatre_id; unique (clerk_user_id,theatre_id); Clerk subject is external.
razorpay_orders / RazorpayOrder	Provider order, customer subject, amount, status, payment and purchase snapshot.	PK id; unique provider_order_id; scalar showtime_id, no JPA FK.
ticket_bookings / TicketBooking	Confirmed paid ticket, QR code, provider IDs, show/seat/amount snapshot, email timestamp.	PK id; unique booking_reference, ticket_code, provider_order_id; scalar showtime_id, no JPA FK.
ride_posts / RidePost	Current ride listing with author, date, route/show, seats/fare, active/version.	PK id; date+active index; required core values; @Version.
ride_join_requests / RideJoinRequest	Passenger request and status for a posted ride.	PK id; FK ride_post_id; unique ride_post/requester.
ride_chat_messages / RideChatMessage	Conversation message, sender, body and timestamp.	PK id; FK join_request_id; index request/time; body column length 1200.
bookings / Booking	Earlier/alternate booking lifecycle with Account and Showtime references.	PK id; unique public_reference; FKs customer and showtime; status, totalPaise.
carpool_rides / CarpoolRide	Earlier/alternate ride model linked to Account and destination show.	PK id; FKs driver and destinationShowtime; fare, seats, active.

There are parallel concepts: ride_posts is current user-facing board, while carpool_rides is an older/alternate entity; ticket_bookings is the current Razorpay confirmed-ticket record, while bookings is another booking lifecycle entity. Confirm data dependencies before retiring either table.

8. Keys, constraints, validation, and transactions

8.1 Primary keys

Each entity uses @Id and @GeneratedValue(strategy=IDENTITY), producing a database-generated surrogate numeric key. Public booking reference and ticket code are separate UUID-like values with unique constraints so internal sequential IDs do not serve as customer references. MySQL primary keys are unique and not nullable; InnoDB organizes rows around the primary key [5].

8.2 Foreign keys and relationships

Declared JPA relationships indicate intended FKs: screens.theatre_id; showtimes.movie_id/screen_id; showtime_seats.showtime_id; theatre_movies.theatre_id/movie_id; theatre_owner_memberships.theatre_id; ride_join_requests.ride_post_id; ride_chat_messages.join_request_id; bookings.customer_id/showtime_id; carpool_rides.driver_id/destination_showtime_id. InnoDB FK constraints prevent orphan references and enforce parent-child integrity [6].

Important gap: TicketBooking.showtimeId and RazorpayOrder.showtimeId are plain Long properties, not JPA associations, so Hibernate does not generate FKs for these columns from mappings. Clerk user IDs are external subject strings and do not reference accounts. Add FKs only with deliberate history-retention and deletion rules.

8.3 Declared uniqueness and indexes

- accounts: unique email and mobile.
- bookings: unique public_reference.
- screens: unique (theatre_id,name).
- showtime_seats: unique (showtime_id,seat_code).
- theatre_movies: unique (theatre_id,movie_id).
- theatre_owner_memberships: unique (clerk_user_id,theatre_id).
- ride_join_requests: unique (ride_post_id,requester_clerk_user_id).
- ticket_bookings: unique booking_reference, ticket_code and provider_order_id.
- razorpay_orders: unique provider_order_id.
- Indexes: showtimes.starts_at; ride_posts(ride_date,active); ride_chat_messages(join_request_id,created_at).

Unique constraints protect against duplicate inserts even when requests race. For nullable values, MySQL unique-index NULL behavior differs from ordinary duplicate-value behavior; required identifiers should be NOT NULL as well as unique where semantically mandatory [5].

8.4 NOT NULL and validation layers

The entities include explicit nullable=false on numerous ticket/order and ride fields; other JPA fields use defaults. Jakarta Bean Validation annotations in entities and request records include @NotBlank, @NotNull, @Min, @Size, @Pattern and @Email. Controllers invoke @Valid for many DTOs. Application rules additionally check distinct seat count, accepted seat syntax, server-side pricing, showtime availability, date/departure windows, owner-screen assignment and future dates. These are not automatically SQL CHECK constraints.

The scanned entity files have no explicit @Check annotations. So the code does not establish declared SQL CHECK constraints for positive amounts, seat counts, duration or date rules. MySQL supports enforced CHECK constraints on row-local expressions [8]; consider adding them as defense in depth. Cross-row rules, such as whether two payments race for the same seats, need transactional logic.

8.5 Transactions and seat safety

Order creation and verification are @Transactional. On verification, the backend validates the provider payment, requests a lock on selected seat records, rechecks existence/availability, marks seats booked, marks order captured, and inserts ticket record. A DB failure can roll back those database changes. The external payment call is not part of the same ACID transaction; a lost response can leave provider/database states temporarily divergent. InnoDB transactions provide atomicity/consistency/isolation/durability within the database [9], not across Razorpay and the app.

9. Normalization review and expansion toward BCNF

9.1 Accurate conclusion from the source

The schema has a 3NF-oriented structure: core concepts have their own tables; venue/movie many-to-many is represented by theatre_movies; per-show seats are separate rows; memberships and ride requests are separate link/lifecycle records. But normal forms depend on functional dependencies and candidate keys, not merely table names. From JPA annotations alone, the complete schema cannot be formally certified as 3NF or BCNF.

9.2 Normal forms in simple language

- **1NF:** one value per field, no lists packed into a cell, rows uniquely identified. Most core data is scalar. However ticket/order seats are comma-separated strings, not relational seat rows; this is a denormalized snapshot.
- **2NF:** each non-key field depends on the whole candidate key. Since most tables use one-column surrogate PKs, partial dependency on a composite PK is not obvious. Link-table business uniqueness still matters.
- **3NF:** descriptive facts depend on the key, the whole key and nothing but the key. Splitting Movie, Theatre, Screen and Showtime reduces repeated mutable descriptions. Ticket snapshots repeat display facts intentionally for history.
- **BCNF:** every determinant is a candidate key. To verify this, document all dependencies and business keys, then assess each determinant; the repository contains no formal dependency specification.

9.3 Historical snapshot versus accidental duplication

TicketBooking copies movie, theatre, show, seats, amount and provider details so a later catalogue edit does not rewrite what the customer purchased. RazorpayOrder captures similar fields before final verification. This is intentional audit/history denormalization. Make it explicit by naming snapshot columns and preserving amounts/currency/seat prices as sold.

Potential model overlap is separate from normalization: Booking/TicketBooking and CarpoolRide/RidePost describe similar domains. Account is separate from Clerk subjects. Before removing or merging these, trace references, existing rows and migrations.

9.4 Expansion steps

- Document functional dependencies and candidate/business keys; inspect MySQL SHOW CREATE TABLE and information_schema to validate generated DDL.
- Add ticket_booking_seats(ticket_booking_id, seat_code, unit_price_paise), FK to ticket_bookings, unique(ticket_booking_id,seat_code). Keep immutable title/venue/date/price snapshots on the ticket for historical display.
- Add FKs for scalar showtime IDs only if retention policy supports them; prefer deactivating shows over deleting after tickets exist.
- Choose one canonical booking model and ride model after tracing use; migrate existing rows and consider compatibility views before retirement.
- Represent multi-user venue ownership as a membership/link entity with theatre/user, role, state, effective dates, approver and audit timestamps.
- Continue storing INR in integer paise. If adding tax/fees/discounts, persist auditable line items and currency rather than recomputing old orders from current prices.
- Add appropriate database CHECK constraints for positive money, duration > 0 and seats >= 0; retain transactions for cross-row inventory rules.
- Define lifecycle and delete policies for every FK, plus retention and privacy policy for tickets and ride chat.

Normalization does not mean removing all repeated values. It means preventing update/insert/delete anomalies while keeping deliberate snapshots when historical accuracy requires them. MySQL supports PK, UNIQUE, FK and CHECK constraints alongside application validation [5][6][8].

10. MySQL adoption and operations

10.1 Configuration

Set backend environment variables in the same terminal session that launches Maven. A local URL has the form `jdbc:mysql://localhost:3306/picturehouse?serverTimezone=UTC`; driver `com.mysql.cj.jdbc.Driver`; use a dedicated least-privilege `picturehouse_app`, not root. Set DATABASE_URL, DATABASE_USER, DATABASE_PASSWORD, DATABASE_DRIVER. Never put credentials in source, GitHub or frontend VITE variables. Restart the backend after changes. Check startup logs/runtime URL; Workbench connection success alone is not enough.

10.2 Recommended MySQL setup

- Use MySQL 8.4 LTS and InnoDB; choose utf8mb4. Connect Workbench to the intended localhost:3306 instance.
- Create the project schema and an application account scoped to localhost with only required rights. Keep a separate administrative account.
- Run the Java API with MySQL environment settings; confirm selected schema/version with a SQL query and confirm new application rows appear there.
- Back up H2 data before switching. H2 files and MySQL are separate databases; changing the URL does not migrate old records.

10.3 Migration management

Hibernate ddl-auto=update helps a prototype evolve a local schema, but it does not provide reviewed, repeatable migration history. Adopt Flyway or Liquibase before production. Check in a baseline schema and numbered future changes. In production, prefer ddl-auto=validate so app startup checks compatibility without silently rewriting schema.

10.4 Workbench verification queries

Use `SHOW CREATE TABLE showtime_seats;`, `SHOW CREATE TABLE theatre_owner_memberships;`, and `SHOW CREATE TABLE ticket_bookings;`. Query information_schema.table_constraints, key_column_usage and statistics for all constraints/indexes. Compare the database/schema shown by Workbench with the running backend URL before interpreting results.

11. Security, reliability and limitations

11.1 Implemented defenses

- Bearer JWT validation checks signature, issuer/time, and configured origin claim.
- Razorpay secret remains on backend; amount is recomputed server-side; signature and provider capture details are verified.
- Owner requests are checked against approved membership for the specific theatre; authentication alone does not imply venue ownership.
- Seat uniqueness, locking and transactional writes mitigate duplicate inventory booking.
- Email HTML escapes user-visible content before inclusion.

11.2 Limitations before production

- Development seed records are fictional; real listing and seat/pricing accuracy requires theatre data and a controlled publishing process.
- No versioned database migrations; H2 and MySQL behavior may differ; inspect generated DDL and run database-specific concurrency verification.
- Payment webhook reconciliation, idempotency workflow and durable refund status are not documented as implemented.
- QR is generated but gate redemption/single-use/expiry flow is future work.

- Ticket seats are a CSV field and showtime IDs in snapshot tables are not declared FKs.
- Manual ownership approval is the current workflow. A theatre Gmail address is not ownership verification.
- Personal data includes email, Clerk identifiers, ride chats and payment references. Define retention/deletion/export, access audit and backup protection.
- Catalogue uses Asia/Kolkata LocalDateTime; define timezone handling across DB, API, emails and payment events before multi-city expansion.
- Production needs structured, redacted logs, request IDs, metrics, alerts and error reporting. Do not log secrets or unnecessary PII.
- Pin package versions and use lockfile-based CI installs for reproducibility.

12. Recommended next steps

Priority	Action and why
1. Confirm active DB	Check DATABASE_URL in Maven launch session and query selected schema. Back up H2 before moving data.
2. Baseline MySQL	Create schema and app user; run API against MySQL; inspect SHOW CREATE TABLE for all declared FKs, uniques and indexes.
3. Add migrations	Adopt Flyway/Liquibase and baseline; production ddl-auto=validate.
4. Resolve overlap	Trace Booking vs TicketBooking and CarpoolRide vs RidePost; decide canonical models with a migration plan.
5. Harden payment	Add idempotency, signed webhooks, reconciliation, refund lifecycle, durable email outbox, concurrency checks.
6. Verify owner onboarding	Use trusted business-domain verification/documents and audited membership/recovery process; do not rely on Gmail naming alone.
7. Improve integrity	Add ticket seat detail rows, justified FKs/CHECKs, delete policy, and formal functional-dependency notes.
8. Release controls	Pin dependencies; secret store; separate test/live credentials; TLS/CORS/profiles, backups, monitoring and privacy operations.
9. Replace demo data	Load theatre-verified films, shows, screens, seat maps and pricing; review with theatre operators before sales.

Appendix A. Logical ER diagram

This logical view emphasizes current relationship-bearing entities. Each 1:N label means a parent may have several child rows. JPA mappings are shown; generated MySQL names, column types and constraints must be confirmed from actual DDL. Ticket/order showtime IDs are scalar values in current code, so no FK line is drawn to them.

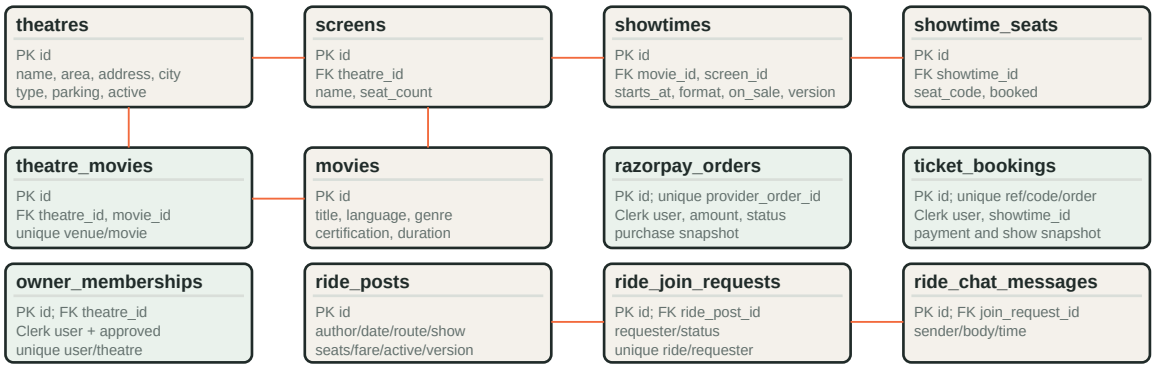

Figure 2. Core cinema, payment, owner and ride-board model. accounts, bookings and carpool_rides are omitted for legibility; see table inventory. ticket_bookings.showtime_id and razorpay_orders.showtime_id are not declared JPA foreign keys.

A.1 Relationship summary

Parent	Relationship	Child / meaning
theatres	1:N	screens: a venue contains screens.
screens	1:N	showtimes: a screen hosts scheduled shows.
movies	1:N	showtimes: each show belongs to one movie.
showtimes	1:N	showtime_seats: inventory for each show.
theatres + movies	M:N via link	theatre_movies assigns movies to venues.
theatres	1:N memberships	one membership identifies one user/theatre relationship.
ride_posts	1:N	ride_join_requests: passengers request one ride.
ride_join_requests	1:N	ride_chat_messages: messages tied to request conversation.
razorpay_orders	logical unique reference	ticket_bookings stores unique provider order ID, but there is no JPA FK.

Appendix B. Source and research references

B.1 Project files inspected

- package.json and package-lock.json: scripts and frontend dependencies.
- backend/pom.xml: Java/Spring versions and dependencies.
- backend/src/main/resources/application.yml: datasource, mail, payment, security, CORS and schema behavior.
- src/App.jsx, src/ClerkAuth.jsx, src/OwnerPortal.jsx, src/RideBoard.jsx, src/RideChat.jsx, src/api.js: UI and API clients.
- backend/src/main/java/in/picturehouse/jaipur/api/: catalog, auth, owner, payment, ride and ticket routes.
- backend/.../domain/: entities, constraints, enums and domain methods.
- backend/.../repository/: Spring Data query declarations and seat-lock query.
- backend/.../config/: security, development data seeding and WebSocket config.
- backend/.../integration/: Razorpay, payment/delivery abstractions and email/QR.
- README.md and docs/: prototype limits and local configuration guides.

B.2 External technical references

[1] React: Describing the UI; Adding Interactivity
https://react.dev/learn/describing-the-ui
https://react.dev/learn/adding-interactivity

[2] Vite: Why Vite; Getting Started
https://vite.dev/guide/why
https://vite.dev/guide/

[3] Spring Data JPA reference
https://docs.spring.io/spring-data/jpa/reference/jpa.html

[4] Spring Security: OAuth 2.0 Resource Server JWT
https://docs.spring.io/spring-security/reference/servlet/oauth2/resource-server/jwt.html

[5] MySQL 8.4: PRIMARY KEY and UNIQUE constraints
https://dev.mysql.com/doc/refman/8.4/en/constraint-primary-key.html

[6] MySQL 8.4: Foreign-key constraints
https://dev.mysql.com/doc/refman/8.4/en/create-table-foreign-keys.html

[7] Oracle Java Tutorials: OOP concepts
https://docs.oracle.com/javase/tutorial/java/concepts/

[8] MySQL 8.4: CHECK constraints
https://dev.mysql.com/doc/refman/8.4/en/create-table-check-constraints.html

[9] MySQL 8.4: InnoDB introduction and transaction properties
https://dev.mysql.com/doc/refman/8.4/en/innodb-introduction.html

[10] Razorpay developer documentation
https://razorpay.com/docs/

Research note: official documentation supports general technology behavior. Project-specific claims were read from the current repository; dependency declarations are not claims that a version is the newest available.